

DAT 1기 캡스톤 프로젝트 보고서

기간에 따른 ETF 수익률 예측 딥러닝 모델 분석

: LSTM 과 Transformer를 활용하여

DAT 1기

5팀

김 협

신혜선

이예진

이호영



DAT 1기 캡스톤 프로젝트 보고서

기간에 따른 ETF 수익률 예측 딥러닝 모델 분석

: LSTM 과 Transformer를 활용하여

Analyzing the Deep Learning Model for Forecasting ETF Yields Over Period

: Using LSTM and Transformer

이 보고서를 캡스톤 프로젝트 보고서로 제출합니다.

2023년 06월 06일

DAT 1기

5팀

김 협

신혜선

이예진

이호영



Data Analysis & Technology



DAT 1기 캡스톤 프로젝트 보고서

기간에 따른 ETF 수익률 예측 딥러닝 모델 분석

: LSTM 과 Transformer를 활용하여

국문 요약

본 연구의 목표는 기간에 따른 ETF 수익률 예측 모델을 개발하고, 그 예측성을 평가하는 것이다. 여기서 수익률 예측 모델은 장기 의존성이 개선된 모델인 LSTM과 Transformer를 활용하며, 투자 기간에 따른 예측 모델의 성과를 평가 및 비교한다. 본 연구에서 투자 기간에 대한 기준은 1개월을 단기, 3개월을 중기, 6개월을 장기로 설정한다. 또한, 주가의 등락 추세에 따라 다섯 가지 기간을 설정하고 그 기간에 따른 모델별 예측력 또한 비교한다. 모든 모델 학습 및 예측 단계에서는 종가 feature만을 사용하여, 이전 연구와의 비교 가능성과 이후 연구로의 확장성을 도모한다.

Abstract

The purpose of this study is to develop an ETF yield prediction model over time and evaluate its predictability. Here, the yield prediction model utilizes LSTM and Transformer, which are models with improved long-term dependence, evaluates and compares the performance of the prediction model according to the investment period. In this study, the criteria for the investment period are set as short-term for 1 month, medium-term for 3 months, and long-term for 6 months. In addition, five periods are set according to the fluctuation trend of stock prices, and the predictive power of each model according to the period is also compared. In every model learning and prediction stages, only the closing price feature is used to promote comparability with previous studies and scalability to future studies.



목 차

제 1장 연구 배경 및 목표	5
제 2장 자료 및 연구방법	6
제 1절 데이터 소개	6
제 2절 LSTM	8
제 3절 Transformer	9
제 3장 실험	12
제 1절 Feature 비교 실험	14
제 2절 하이퍼파라미터 튜닝 실험	16
제 3절 기간별 예측 실험	19
제 4절 실험의 한계	28
제 4장 결론 및 한계점	30
제 1절 후속 연구	31
참고문헌	33



표 목차

[Table 1]	15
[Table 2]	16
[Table 3]	17
[Table 4]	18
[Table 5]	19
[Table 6]	21
[Table 7]	22
[Table 8]	22
[Table 9]	23
[Table 10]	23
[Table 11]	24
[Table 12]	24
[Table 13]	25
[Table 14]	26
[Table 15]	26
[Table 16]	26



그림 목차

[Figure 1]	7
[Figure 2]	8
[Figure 3]	9
[Figure 4]	11
[Figure 5]	13
[Figure 6]	14
[Figure 7]	15
[Figure 8]	17
[Figure 9]	18
[Figure 10]	20
[Figure 11]	21
[Figure 12]	21
[Figure 13]	22
[Figure 14]	22
[Figure 15]	23
[Figure 16]	24
[Figure 17]	24
[Figure 18]	25
[Figure 19]	25



[Figure 20]	26
[Figure 21]	27
[Figure 22]	27
[Figure 23]	28
[Figure 24]	29
[Figure 25]	30

수식 목차

[식 1]	12
[식 2]	12
[식 3]	13
[식 4]	28



제 1 장 연구 배경 및 목표

주식 수익률을 예측하는 것은 매우 어렵다고 알려진 분야임에도 불구하고 오랜 기간 꾸준히 시도되어 왔다. 또한 최근 딥러닝을 사용하는 퀀트 투자자들 또한 증가하는 추세를 보이고 있다. 컴퓨터는 복잡한 계산을 사람보다 빠르고 정확하게 수행할 수 있다. 분야다 **김동건 외. (2021)**에 의하면 이러한 특징을 반영하여 상당한 양의 변수가 존재하는 다양한 분야에 딥러닝을 활용한 연구가 활발히 이뤄지고 있고, 주가 예측 또한 그 중 한 분야다. 시계열 데이터를 딥러닝 모델을 통해 예측에는 일반적으로 RNN(Recurrent Neural Network) 기반의 모델이 활용되고, *VASWANI et al. (2017)*에서는 Attention 기법만을 활용한 모델인 Transformer의 활용 또한 시도되고 있다. 하지만 RNN 기반 모델인 Vanila RNN, LSTM(Long Short Term Memory)¹, GRU(Gated Recurrent Unit)와 Attention 기법만을 활용한 Transformer의 주가 예측 성과를 비교한 연구는 부족하다. 또한, **김수현. (2020)**에서 LSTM 모델을 우리나라 주식시장에 적용하여 포트폴리오 전략을 수립하고, 이 전략이 가격 결정 모형에 유의성을 주는지 확인한 연구는 존재하지만, 투자 기간에 따른 ETF 수익률 예측 모델의 성과를 비교한 연구는 부족한 실정이다.

ETF (Exchange Traded Fund)의 보수비용은 일반 펀드에 비해 저렴하고, 증권거래세가 부분적으로 면제되어 매매 비용 또한 저렴하다. 또한 가격, NAV(Net Asset Value), 벤치마크 지수를 실시간으로 확인할 수 있으며, 월별 보유종목을 공시하는 일반 펀드와는 달리 설정과 환매를 위해 보유 종목이 PDF(Portfolio Deposit File) 형태로 일별 공시된다는 점에서 그 투명성이 높다.

본 연구에서는 일반적으로 많이 활용되고 있는 시계열 예측 모델 LSTM 을 통해 종가 수익률을 예측한다. 또한, LSTM 과의 비교를 위해 Transformer 를 활용한 종가 수익률 예측 모델 또한 학습한다. 이후, 두 모델을 비교 분석하여 각 매매 포지션에 따른 두 모델의 성능을 평가 및 비교한다. 실험 기간은 2013년 1월 4일부터 2023년 1월 4일까지이다. 다양한 외부 요인과 개별적인 사건에 영향을 더 많이 받는 개별 주식에 아닌 ETF 를 통해 실험함으로써 예측의 불안정성과 노이즈를 줄일 수 있다. 학습을 위한 데이터는 KOSPI 200 지수를 추종하는 ETF 상품인 KODEX 200 종가 데이터다. 두 모델의 학습을 진행한 후, 각 모델을 통해 KODEX 200 종가 수익률을 예측한다. 이 때 딥러닝 프레임워크로는 PyTorch 를 이용한다.

매매 포지션은 일반적으로 투자 기간, 투자 성향, 투자금과 같은 요소에 따라 다양하게 분류된다. 본 연구는 투자 기간과 가격 변동성에 따른 ETF 수익률 예측 모델의 개발과 비교 분석을 통해 효과적인 투자 전략의 도출과 투자 전략에 따른 합리적인 예측 모델 선정을 목표로 한다. 투자 기간은 1, 3, 6개월로 나누어 수익률을 예측하고 이에 따른 매매 전략을 수립한다. 본 연구 결과를 통해 기간에 따른 투자 결정에

¹ HOCHREITER, Sepp; SCHMIDHUBER, Jürgen. Long short-term memory. *Neural computation*, 1997, 9.8: 1735-1780.



딥러닝 모델을 활용방안으로 제시될 수 있는지 유의성을 살펴본다.

제 2장 자료 및 연구방법

제 1절 데이터 소개

본 연구에서 사용한 데이터는 KOSPI 200지수 추종 ETF인 KODEX 200이다. 이 데이터는 API를 활용해 KRX데이터를 크롤링하여, 2013년 1월 4일에서 2023년 1월 4일까지의 기간을 일별로 수집하였다.

본 데이터는 주식 가격 관련 열로 저가, 고가, 시가, 종가 열을 포함하고 있다. 주식 가격 관련 열과 NAV 열은 서로 선형 관계를 가지며, 매우 높은 상관관계를 갖고 있는 것을 Figure2를 통해 확인할 수 있다. 높은 상관관계를 갖는 열을 동시에 포함하는 모델은 다중공선성을 비롯한 다양한 문제를 야기하기에, 위 5가지 열 중 하나의 열만을 선정하여 모델을 학습할 필요가 있다. 김수현. (2020).에서는 LSTM을 적용하여 일 단위로 개별 종목의 종가를 이용해 매매 시점 결정 및 수익률 평가를 연구한 바 있다. 따라서, 본 연구에서는 기존 연구인 <LSTM 기반 모형의 주식시장 예측성 분석>과의 비교와 Attention 기법의 특성을 고려하여 기존 연구와 동일하게 종가 열만 활용한다. 또한 종가는 거래일 동안 변동될 가능성이 있는 다른 가격 지표들(시가, 고가, 저가)과 달리 보다 안정적이고 확정된 가격으로 간주된다는 장점이 있다. 종가를 활용함으로써 추후 미국 주식 시장으로의 확장성도 기대할 수 있다.



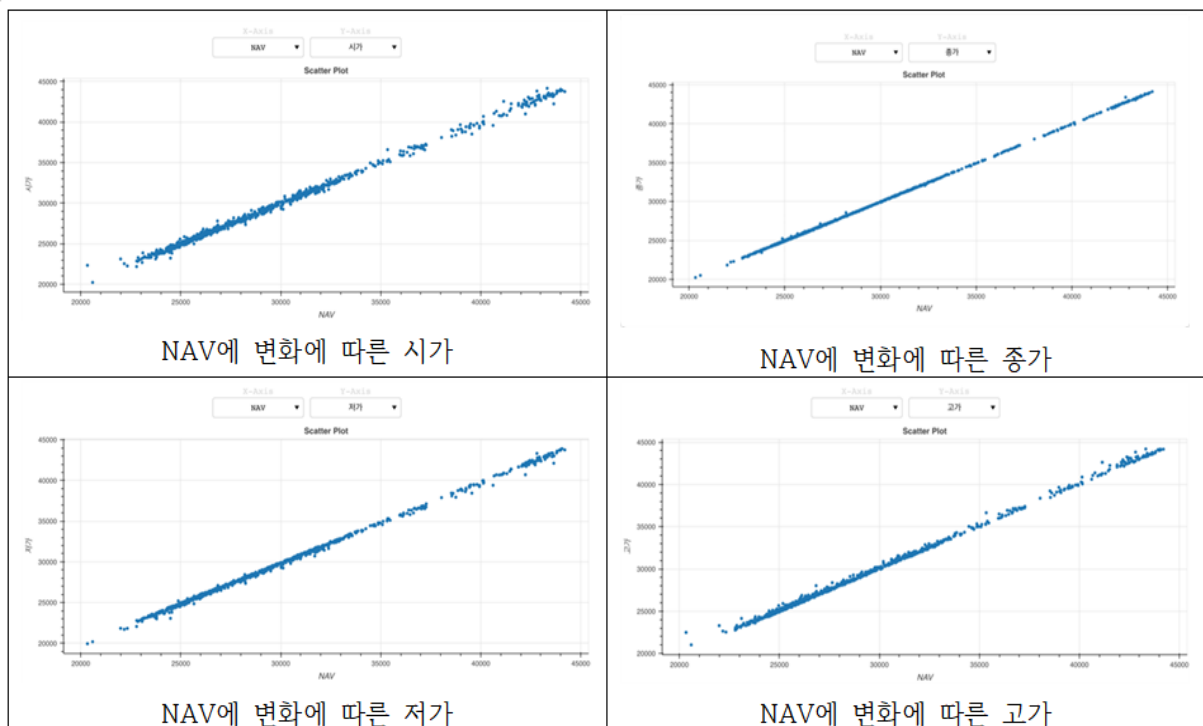


Figure 1. NAV 변화에 따른 가격 데이터들의 유사성

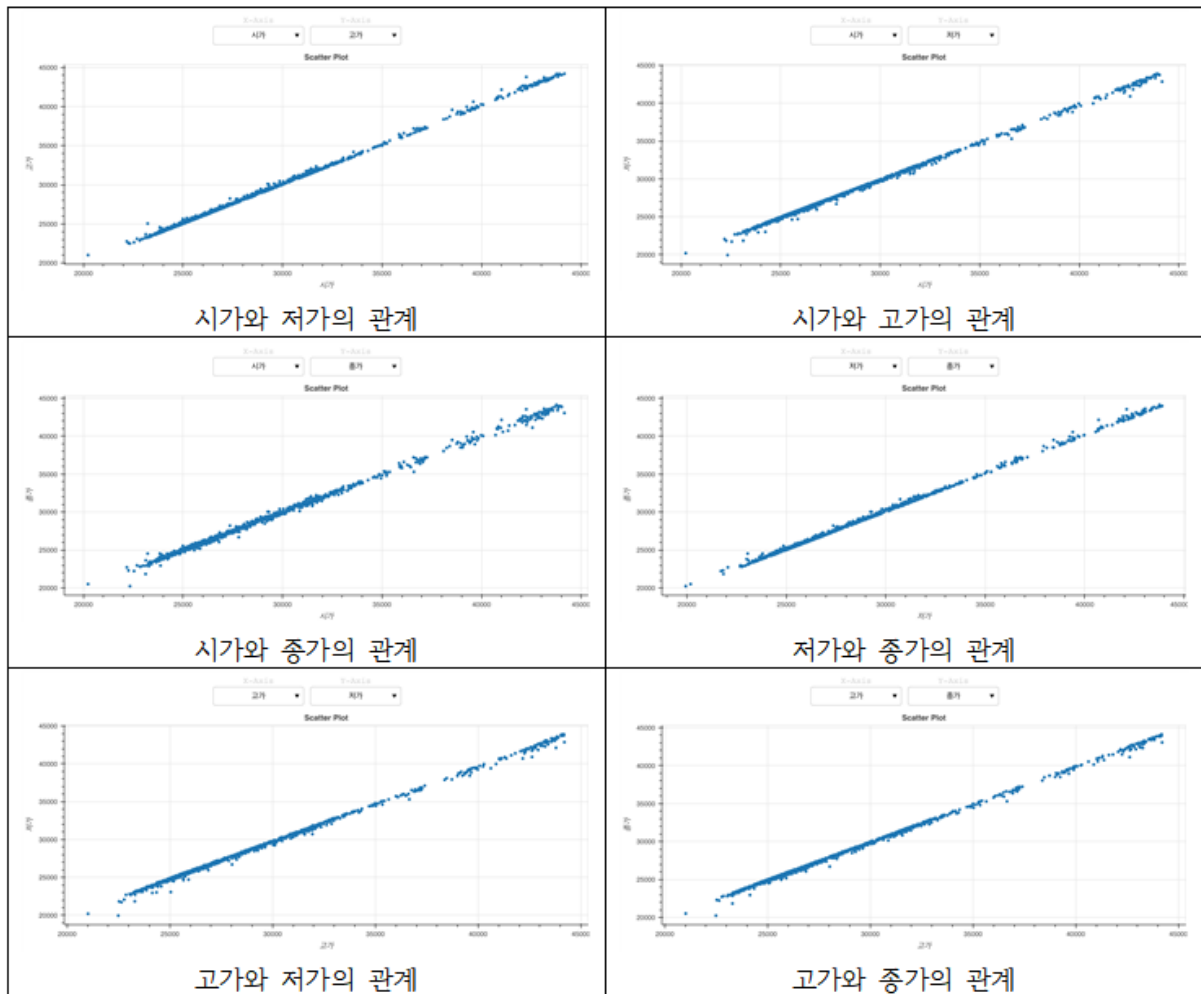


Figure 2. 각 가격 데이터들의 관계

제 2절 LSTM

기존의 Vanilla RNN은 역전파로 인해 타임 스텝이 커질 수록 input 데이터가 여러 개의 layer를 지나며 기울기 소실 또는 기울기 폭발 문제가 발생한다. 이는 장기 의존 관계 학습에 어려움으로 이어져, 금융 시계열 데이터를 분석하는 데에 적절하지 않다. 따라서, *HOCHREITER et al. (1997)*에서는 RNN에 선별적 기억능력 보유를 목적으로 게이트를 추가하여 이러한 문제를 보완한 LSTM 모델을 제안하였다. LSTM의 구성은 다음과 같다.

LSTM은 RNN의 변형 딥러닝 모델로 Vanila RNN과 같은 체인 구조로 되어 있으며, 내부적으로 셀

(memory cell)과 게이트(gate)라는 추가적인 구조를 사용하여 작동한다. LSTM 내부에는 현재의 입력 정보를 얼마나 Cell state에 반영할지 결정하는 역할의 Input gate, 이전 셀 값을 얼마나 유지할지 즉 불필요한 정보를 결정하는 역할의 Forget gate, 최종적으로 어떤 정보를 output으로 내보낼 지 결정하는 역할의 Output gate, 이전 시점의 정보가 변하지 않고 그대로 흐르도록 하는 역할의 Cell state가 존재한다. LSTM은 장기 의존성 문제를 해결하기 위해, 셀과 게이트 구조를 사용하여 선별적 기억 능력을 갖추고 있다. 이를 통해 LSTM은 긴 시퀀스에서 발생하는 기울기 소실 또는 기울기 폭발 문제를 완화하면서, 효과적인 시퀀스 모델링이 가능해진다. 따라서, LSTM은 주식 예측뿐만 아니라, 자연어 처리, 음성 인식, 기계 번역 등 다양한 시퀀스 데이터 처리 작업에 많이 활용되고 있다. LSTM에서는 셀 단위로 정보가 처리되어 다음 셀로 넘어가는데, 이에 대한 간단한 구조는 Figure 3과 같다.

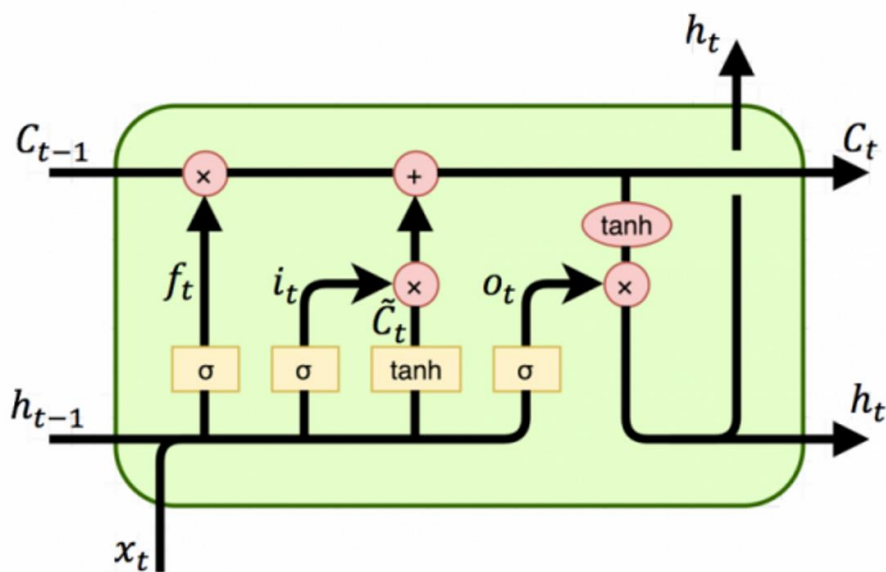


Figure 3. LSTM의 구조

현재 시점의 데이터 x_t 와 이전 시점의 은닉층 h_{t-1} 이 각각 forget gate와 input gate의 입력값이 된다. Forget gate에서 불필요한 정보가 결정되어 남은 정보와 input gate에서 처리되어 넘어온 정보가 새로운 셀을 구성하게 된다. 이렇게 처리된 정보와 output gate에서 독립적으로 x_t 와 h_{t-1} 를 처리하고 은닉층 h_t 를 결정하여 출력한다.

제 3절 Transformer

인코더-디코더 구조로 구성된 기존의 Seq2seq (Sequence-to-Sequence) 모델은 모든 입력 시퀀스를 순차적

으로 입력 받아서 컨텍스트 벡터 (Context vector)라는 하나의 벡터를 생성한다. 이 과정에서 병목현상이 발생한다는 문제점을 지닌다. 또한 기존의 Vanilla RNN에 비해 개선이 되었으나 여전히 장기 의존성 문제가 존재한다. 2017년 구글에서는 이러한 단점들을 보완한 Transformer라는 모델을 제시하였다. 이 모델은 오직 Attention으로만 기존의 Seq2seq의 구조인 인코더-디코더를 구현한 모델이다. Self-Attention Mechanism을 도입하여 입력 시퀀스 내의 모든 단어 쌍 간의 상호 작용을 계산하여, 각 단어의 유의성(Attention Score)을 학습한다. 이를 통해 문장 내에서 중요한 정보에 상대적으로 집중하여 예측할 수 있게 하며, 이론적으로 그 길이 제한이 없어 장기 의존성 문제에서 자유롭다. 또한, 이 방식은 병렬 처리가 가능하고 연산 속도가 빠르기에 기존 Seq2seq가 가지고 있던 병목현상 문제에서 자유롭다. Transformer는 인코더에 입력 문장의 특징을 추출하고, 이를 디코더의 입력으로 넣어 번역하고자 하는 언어의 단어를 출력한다. Transformer의 매커니즘을 간략히 설명하면 다음과 같다.

피드 포워드 신경망은 두 번의 선형 결합을 진행하고, 활성화 함수로는 ReLU 함수를 적용하였다. 인코더 각 각의 sub-layer에 적용된 Add & Norm에서 Add는 정보의 손실과 모델의 학습을 돕기 위해 sub-layer의 입력과 출력을 잔차연결(residual-connection)로 더하는 것을 의미한다. 이 과정을 통해, 전반적인 학습 난이도를 낮추어 초기 모델 수렴 속도를 높일 수 있으며, Global optima를 찾을 확률이 높아진다. Norm은 sub-layer를 거쳐 잔차 연결까지 모두 마친 뒤에 수행하는 Layer Normalization을 의미한다.

Figure 5의 우측에 해당하는 디코더에서는 각 layer가 Masked Multi-Head Attention, Multi-Head Attention, FFN 3개의 sub-layer로 구성되어 있다. 디코더에서도 마찬가지로 한 문장의 단어들에 대한 Self-Attention을 수행하는데, 이때 디코더가 순차적으로 학습 및 예측을 할 수 있도록 디코더 입력에 대해 i 시점 이후의 단어들을 고려하지 않도록 하는 Masked Multi-Head Attention을 사용한다. 다음으로 디코더의 sub-layer 중 Multi-Head Self-Attention은 위에서 언급한 Self-Attention과 달리, 인코더와 디코더를 연결하는 인코더-디코더 Attention이다. Q, K, V가 모두 동일한 입력 단어 벡터였던 Self-Attention과 달리 인코더-디코더 Attention에서 Query는 이전 디코더 layer의 출력이고, Key와 Value는 최종 인코더 layer의 출력이다. 즉, 인코더 입력과 디코더 입력 간의 유사성을 통해 Attention 결과를 도출하는 작업이다. 디코더의 최종 결과에 linear layer와 softmax를 사용해 최종 단어를 출력한다.

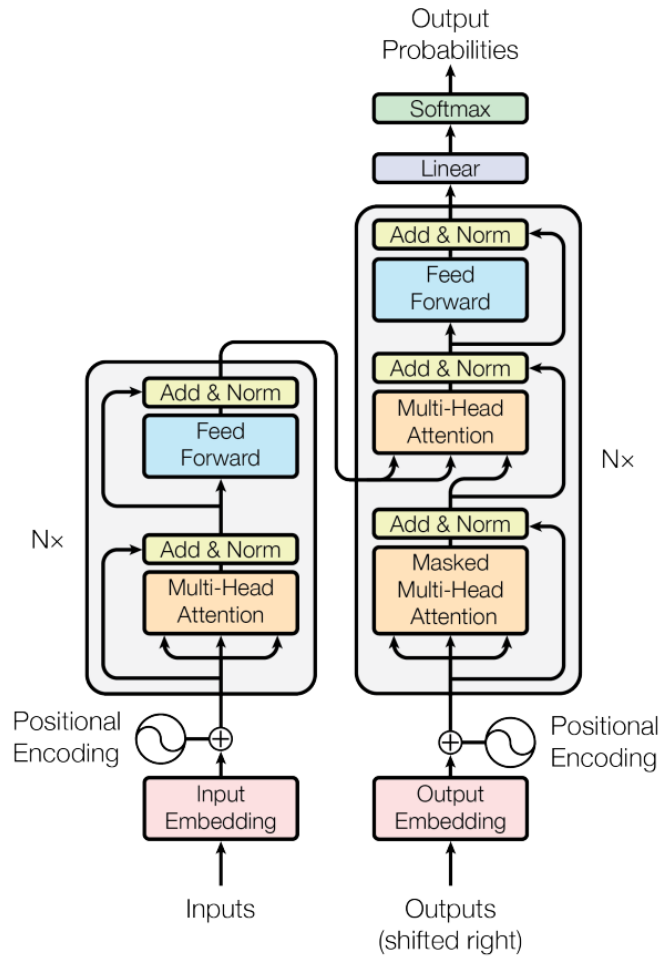


Figure 4. Transformer의 구조

Transformer는 RNN을 사용하지 않고 Attention 기법만을 활용했기 때문에, 입력이 순차적으로 들어 오지 않는다. 따라서, 입력의 위치 정보를 포함하지 않고 있는데 이를 포지셔널 인코딩(Positional Encoding)을 활용하여 문장의 위치 정보를 제공해준다. 포지셔널 인코딩 행렬을 문장 행렬에 더해주어 같은 단어라도 문장 내의 위치에 따라 임베딩 벡터 값이 달라질 수 있도록 한다. 임베딩 벡터가 입력으로 사용되기 전에, 주기적으로 순환하는 $\sin, \cos$ 함수를 이용해 이를 임베딩 벡터에 더해줌으로써 단어의 순서 정보를 더해주는 것이다.

Figure 4의 왼쪽에 해당하는 Transformer의 인코더 layer에는 각 2개의 sub-layer로 구성되어 있다. 이 중 하나는 Multi-Head Self-Attention)이고, 다른 하나는 피드 포워드 신경망(position-wise fully connected feed-forward, FFN)이다. Self-Attention이란 Query(Q), Key(K), Value(V) 사이의 문맥적 관계성

을 추출하는 과정이다. 이 Self-Attention을 병렬적으로 사용한 것이 Multi-head Self-Attention이다. 우선 Query와 Key 사이의 연관성을 계산하기 위해 둘을 내적하여 Attention Score를 구한다. 이때 Query와 Key의 차원이 커져 Attention Score도 커지는 것을 방지하기 위해 $\sqrt{d_k}$ 만큼 나누어주는 Scaled dot-product Attention 연산을 수행한다. 그 다음으로 softmax 활성화함수를 통해 값들을 정규화하고, 보정을 위해 Score행렬과 Value 행렬을 내적해주어 최종적인 Attention 행렬을 얻는다. 각 헤드의 Attention 계산 값을 하나로 합치면 Multi-head Self-Attention의 출력을 얻을 수 있다. 이 과정을 수식으로 표현하면 아래와 같다.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (\text{식 1})$$

Figure 4의 우측에 해당하는 디코더에서는 각 layer가 Masked Multi-Head Attention, Multi-Head Attention, FFN 3개의 sub-layer로 구성되어 있다. 디코더에서도 마찬가지로 한 문장의 단어들에 대한 Self-Attention을 수행하는데, 이때 디코더가 순차적으로 학습 및 예측을 할 수 있도록 디코더 입력에 대해 i 시점 이후의 단어들을 고려하지 않도록 하는 Masked Multi-Head Attention을 사용한다. 다음으로 디코더의 sub-layer 중 Multi-Head Self-Attention은 위에서 언급한 Self-Attention과 달리, 인코더와 디코더를 연결하는 인코더-디코더 Attention이다. Q, K, V가 모두 동일한 입력 단어 벡터였던 Self-Attention과 달리 인코더-디코더 Attention에서 Query는 이전 디코더 layer의 출력이고, Key와 Value는 최종 인코더 layer의 출력이다. 즉, 인코더 입력과 디코더 입력 간의 유사성을 통해 Attention 결과를 도출하는 작업이다. 디코더의 최종 결과에 linear layer와 softmax를 사용해 최종 단어를 출력한다.

제 3장 실험

본 실험에서는 종가 feature만을 활용한 딥러닝 모델의 ETF 수익률 예측성을 분석한다. 이때 종가 feature는 로그 차분을 통하여 로그 수익률을 구한다. 금융 분야에서 수익률을 계산할 때 주로 로그 수익률을 사용한다. 이를 통하여 시간 변화에 따른 스프레드를 시각화 하여 이해 가능하다. 또한, 차분은 시계열의 수준에서 나타나는 변화를 제거하여 추세나 계절성을 감소시키고, 시계열의 평균 변화를 일정하게 만든다.

$$R_a = \log \frac{Price_{t+1}}{Price_t} = \log(Price_{t+1}) - \log(Price_t) \quad (\text{식 2})$$



실험 결과 그래프의 모든 값은 누적 수익률과 그에 대한 예측값이며 예측성을 평가하는 손실 함수로는 평균제곱오차(Mean Squared Error, MSE)를 사용한다. MSE는 오차를 제곱한 값의 평균이다. 이는 예측한 값과 실제 정답과의 차이를 의미하여 MSE 값이 작을수록 성능이 좋은 것이다. 주가와 같은 연속형 데이터를 예측할 때 사용되고, 수식은 아래와 같다. 이때 y_i 는 i 번째 학습 데이터의 정답, $\hat{y}_i$ 는 i 번째 학습데이터로 예측한 값이다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (\text{식 3})$$

슬라이딩 윈도우 알고리즘을 활용하여 시간 종속성을 학습한다. 일정한 타임 스텝을 정하여 윈도우 크기를 정한다. 정해진 윈도우를 이동시키며 시퀀스 데이터를 생성한다. 시계열의 시작 부분에서 정의된 크기의 첫 번째 창을 선택하여 시작한다. 이 창은 입력 시퀀스 역할을 하며 다음 시간 단계 값(창 바로 다음)은 해당 출력 값이 된다. 시계열 끝에 도달할 때까지 정의된 보폭만큼 창을 이동하여 이 프로세스를 반복한다. 이렇게 하면 여러 시퀀스 데이터가 생성된다. 여러 시퀀스 데이터로 구성된 데이터 셋을 전체적으로 학습한다.

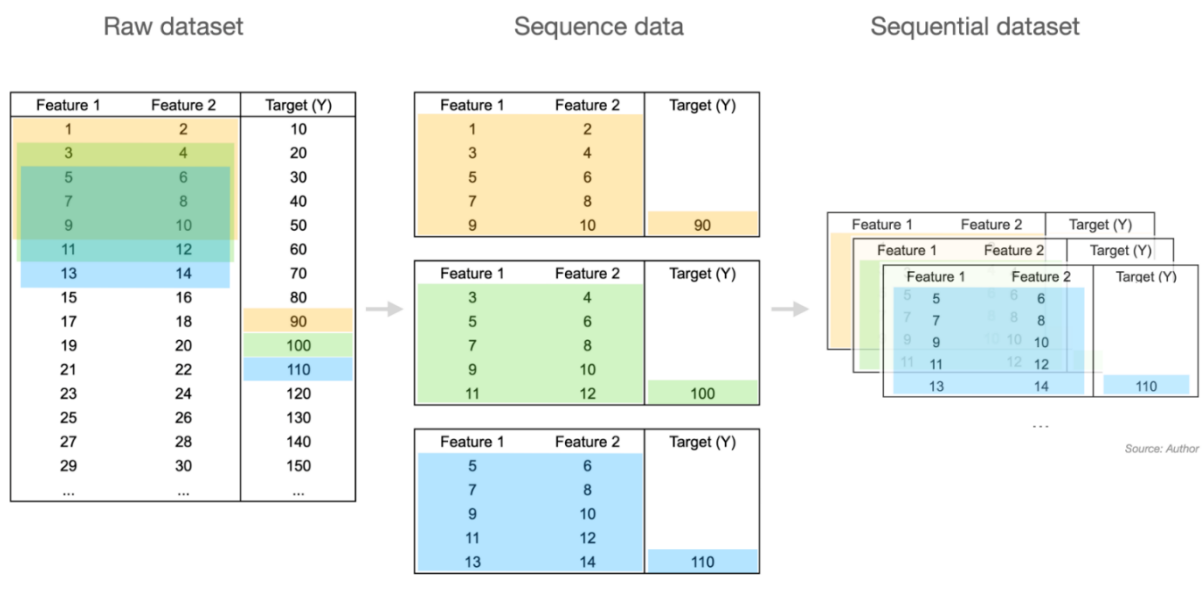


Figure 5. 슬라이딩 윈도우 알고리즘

제 1절 Feature 비교 실험

LSTM과 Transformer 두 모델에서 가격 데이터들의 변수들로 예측을 해 본 결과, 앞의 Figure2, 3에서 보여지는 것과 마찬가지로 가격 데이터들의 유사성이 드러났다. Figure 5는 LSTM모델을 사용한 실험에서의 종가, 시가, 고가, 저가 데이터 각각의 누적 수익률 예측 값과 실제 값을 나타내며, 각 결과의 loss는 Table1에 해당한다.

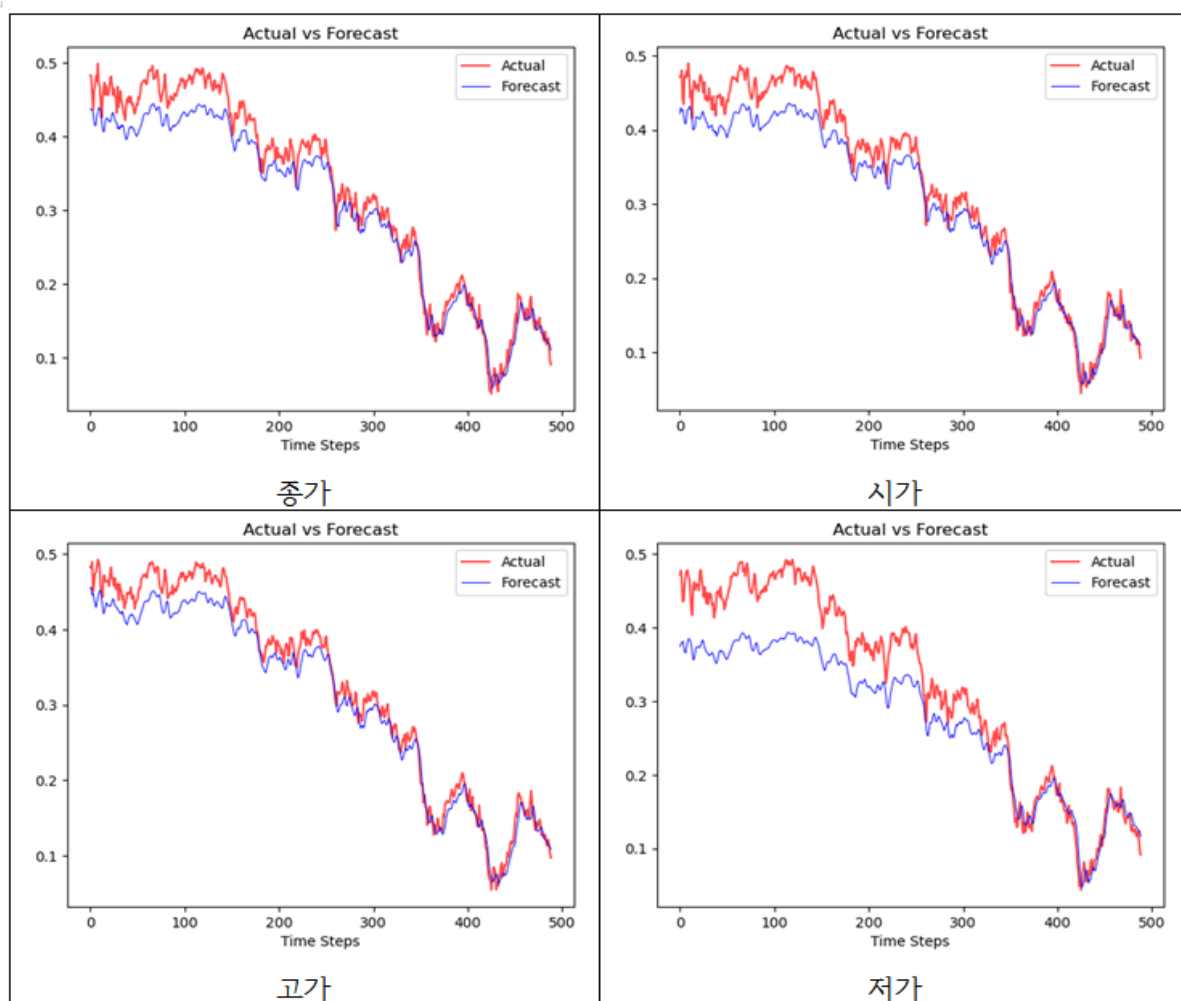


Figure 6. 종가, 시가, 고가, 저가의 LSTM 누적 수익률 예측 값과 실제 값

	종가	시가	저가	고가
loss	0.0008364	0.009225	0.0032975	0.0005525

Table 1. LSTM의 각 가격 데이터별 loss

Figure 6은 Transformer모델을 사용한 실험에서의 종가, 시가, 고가, 저가 데이터 각각의 누적 수익률 예측 값과 실제 값을 나타내며, 각 결과의 loss는 Table2에 해당한다.

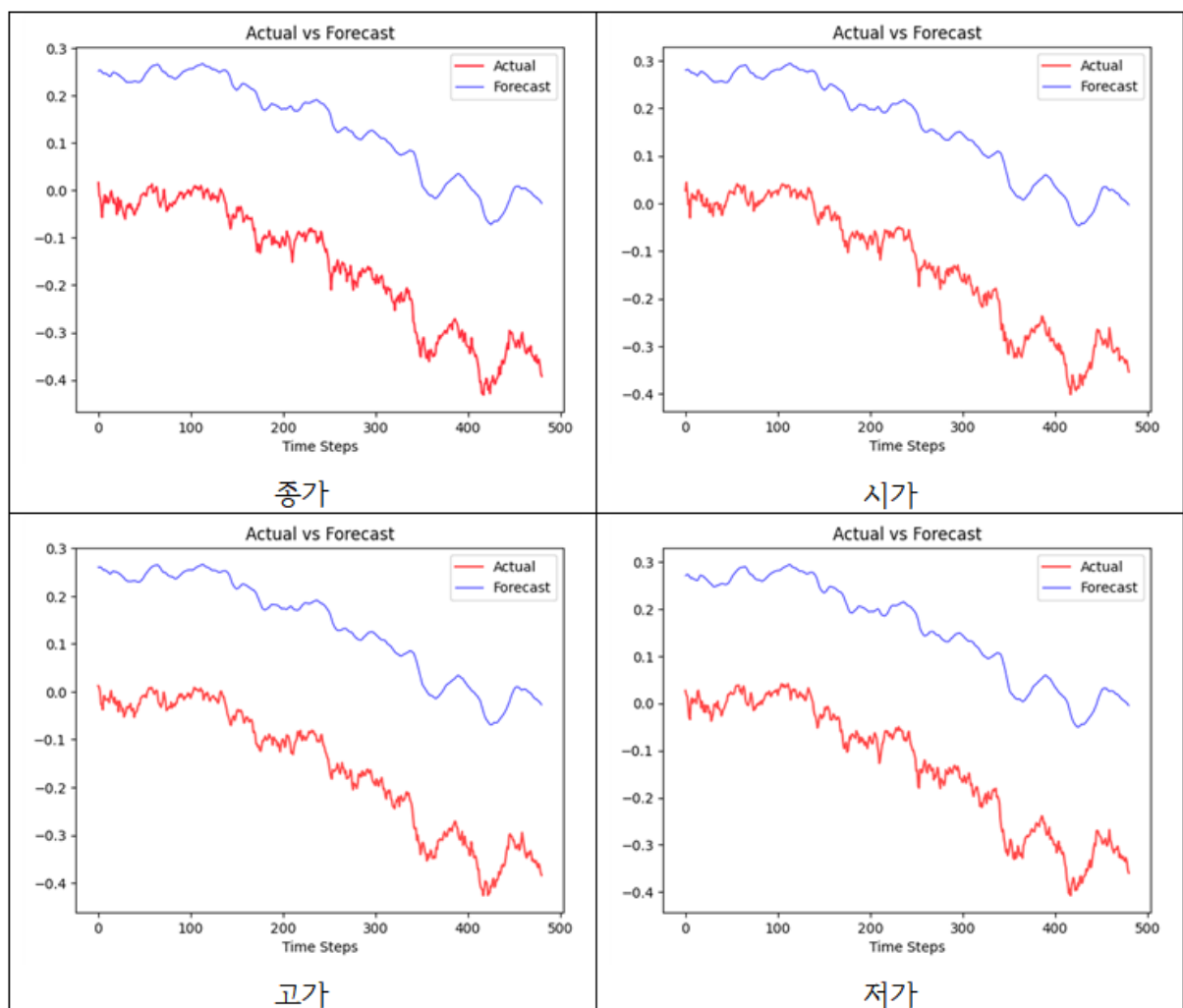


Figure 7. 종가, 시가, 고가, 저가의 Transformer 누적 수익률 예측 값과 실제 값

	종가	시가	고가	저가
loss	0.04710	0.04496	0.04700	0.04522

Table 2. Transformer의 각 가격 데이터별 loss

위 결과를 통해 가격 데이터 변수 간 예측 그래프의 형태와 그 성능이 크게 차이가 나지 않음을 확인하였다. 따라서 본 연구에서는 기존 연구와의 연결성, 추후 연구 확장성을 고려하여 가격 데이터 변수 중 종가를 채택하여 실험을 진행하였다.

제 2절 하이퍼파라미터 튜닝 실험

- LSTM

본 연구의 LSTM 모델 학습을 위한 대표적인 하이퍼파라미터(Hyperparameter)에는 num_layer, dropout, hidden size, batch size가 있다. 하이퍼파라미터란 모델 학습을 관리하는 데 사용하는 외부 구성 변수로, 모델의 성능과 정확도가 정해지는 데에 결정적인 역할을 한다. 하이퍼파라미터에 설정된 규칙 혹은 최적 값이 정해진 것은 없어 모델을 학습하기 전 수동으로 설정해야 한다. 이때 최적의 하이퍼파라미터 세트를 찾기 위해 하이퍼파라미터 튜닝을 진행한다. num_layer는 인코더와 디코더 층의 개수이고 LSTM에서는 단일 레이어를 사용했다. drop out은 서로 연결된 layer에서 뉴런을 제거할 확률을 정하여 과적합(overfitting)을 방지하고 일반화 기능을 향상시키는 데 도움이 되는 하이퍼파라미터다. hidden size는 과거 입력에 대한 정보를 유지하고 유지할 정보와 잊어버리거나 업데이트할 정보를 선택적으로 결정하는 메모리 셀 역할을 한다. batch size는 전체 학습 데이터셋을 여러 작은 그룹을 나누었을 때 하나의 소그룹에 속하는 데이터 수를 의미한다. 전체 학습 데이터셋을 작게 나누어 효율적인 학습시간과 학습 안정성을 확보하기 위함이다.

Figure 7의 좌측은 LSTM 모델의 하이퍼파라미터 튜닝 전, 우측은 튜닝 후의 결과를 보여준다. Table 3은 초기 모델의 하이퍼파라미터 설정에서 num_layer를 제외한 하이퍼파라미터 한 개씩 변화를 주어 하이퍼파라미터 튜닝을 진행했을 때의 loss를 보여준다. 아래의 Figure 7과 Table 3을 통해 드러나듯, 하이퍼파라미터 튜닝을 진행하기 전에 비해 진행한 후의 loss가 0.015에서 0.0000051로 개선되었다. 최종 모델은 (D)로, 각 하이퍼파라미터를 다음과 같이 설정하고 후의 실험을 진행하였다. num_layer = 1, drop out = 0.1, hidden size = 10, batch size = 100

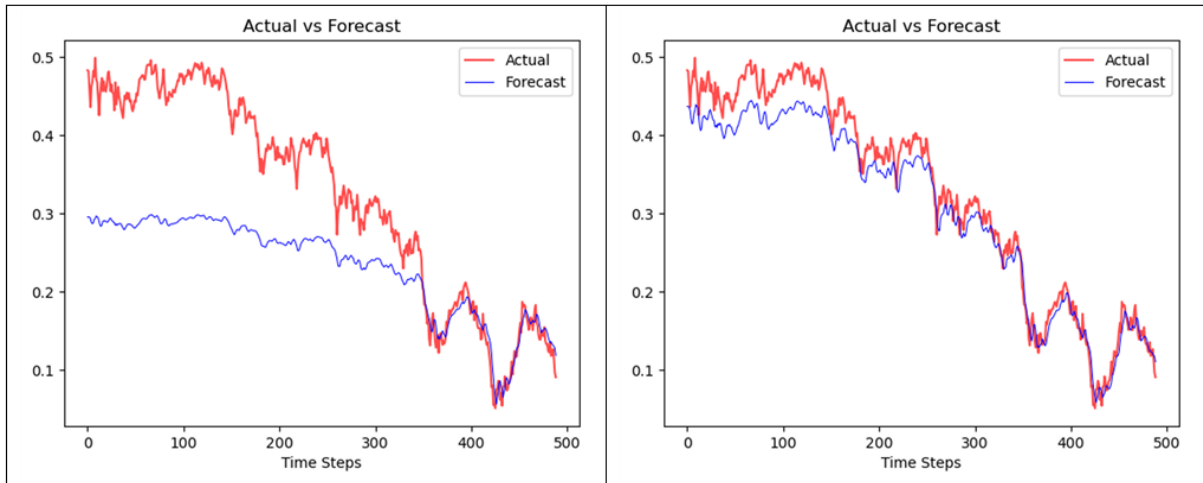


Figure 8. LSTM의 하이퍼파라미터 튜닝 전과 후

	num_layers	drop out	hidden size	batch size	loss
original	1	0.2	5	50	0.015
(A)		0.0			0.0000161
		0.1			0.0000051
		0.2			0.0000091
(B)			5		0.0000092
			10		0.0000051
			100		0.0000069
(C)				50	0.0000052
				100	0.0000051
				200	0.0000833
(D)	1	0.1	10	100	0.0000051

Table 3. LSTM의 하이퍼파라미터 튜닝

- Transformer

본 연구의 Transformer 모델 학습을 위한 대표적인 하이퍼파라미터에는 num_layer, dropout, nhead, batch size가 있다. 여기서 nhead는 multi-head attention의 병렬 헤드의 개수를 의미한다. Figure 8의 좌측은 하이퍼파라미터 튜닝 전, 우측은 튜닝 후의 결과를 보여준다. Table 4는 초기 모델의 하이퍼파라미터 설정에서 하이퍼파라미터 한 개씩 변화를 주어 하이퍼파라미터 튜닝을 진행했을 때의 loss를 보여준다. 아래의 Figure 8과 Table 4를 통해 드러나듯, 하이퍼파라미터 튜닝을 진행하기 전에 비해 진행한 후의 loss

가 0.05267에서 0.00643으로 개선되었다. 최종 모델은 (E)로, 각 하이퍼파라미터를 다음과 같이 설정하고 후의 실험을 진행하였다.

num_layer = 2, drop out = 0.2, nhead = 1, batch size = 100

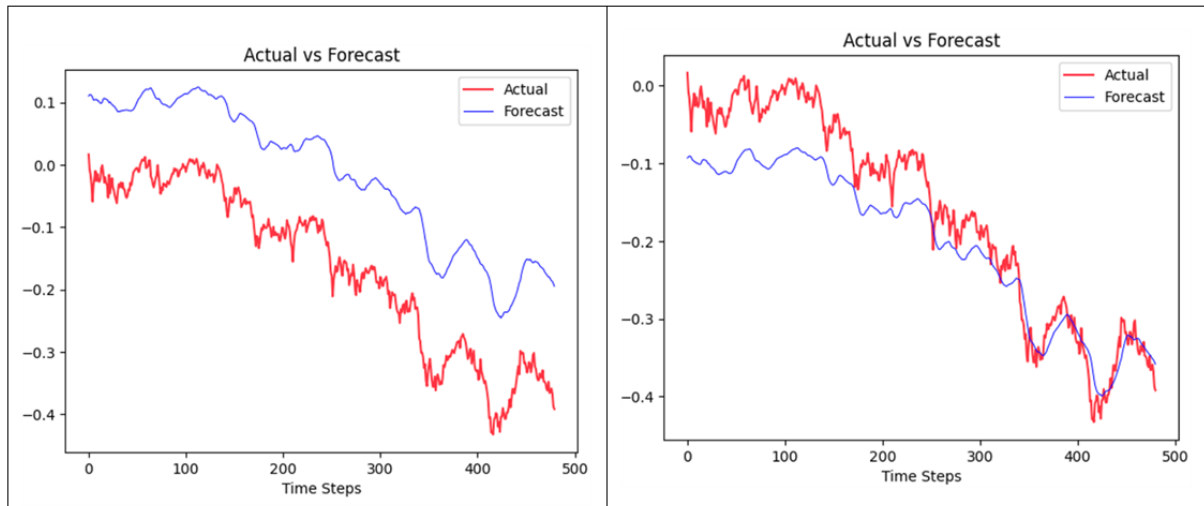


Figure 9. Transformer의 하이퍼파라미터 튜닝 전과 후

	num_layers	drop out	nhead	batch size	loss
original	1	10	10	250	0.05267
(A)	2	0.2	1	100	0.00799
	4				0.2089
	6				0.01745
	8				0.01620
	8				0.01620
(B)	2	0.0	1	100	0.01166
		0.2			0.00246
(C)	2	0.2	1	100	0.00452
			5		0.00465
			25		0.01407
			50		0.00555
(D)	2	0.2	1	100	0.00643
(E)	2	0.2	1	100	0.00643

Table 4. Transformer의 하이퍼파라미터 튜닝

학습률(Learning rate)이란 손실함수 그래프 상에서 이동하게 만드는 보폭의 크기를 의미한다. 이로 인해 모델 구조에 따라 적합한 학습률이 다르고, 학습률을 어떻게 설정하느냐에 따라 예측 결과가 달라진다. 본 연구에서는 적합한 학습률이 각 모델과 학습 기간에 따라 다르다고 판단하여, 모델에 따라 상이하게 설정했다. 또한 학습 가능한 데이터셋의 크기 차이에 따라서도 학습률을 다르게 설정했다. 학습률을 낮게 설정한 경우, local optima²에 빠질 수 있기 때문에 LSTM은 비교적 큰 학습률로 학습을 진행하였고 Transformer는 LSTM에 비해 작은 학습률로 학습을 진행하였다. 아래 Table에서 (A)는 5년 데이터로 학습한 모델의 학습률을, (B)는 4년 데이터로 학습한 모델의 학습률이다.

	(A)	(B)
LSTM	0.01	0.005
Transformer	0.0001	0.00005

Table 5. LSTM과 Transformer의 학습률

제 3절 기간별 예측 실험

본 연구의 실험과정을 간단히 설명하면 다음과 같다. 주가의 등락 추세에 따라 예측 시점을 다음과 같이 설정하였다.

- S1(2020년 12월 21일): 상승 추세에서의 급상승장이 시작되는 시점
- S2(2020년 03월 20일): 급하락 후 급상승장이 시작되는 시점
- S3(2020년 02월 19일): 급하락장이 시작되는 시점
- S4(2016년 12월 08일): 완만한 상승장이 시작되는 시점
- S5(2018년 06월 05일): 완만한 하락장이 시작되는 시점

여기서 모델의 학습 기간은 S1, S2, S3는 예측 시점으로부터 과거 5년까지 설정하였고, 예측 시기가 이른 S4, S5는 예측 시점으로부터 과거 4년까지 설정하였다. 또한 매매 포지션을 투자 기간을 통해 구분하고 1

² : 함수 값이 인근 점에서 보다는 더 작지만 멀리 있는 점에서 보다는 더 클 수 있는 점

“국소 최적해와 전역 최적해”, MathWorks, <https://kr.mathworks.com/help/optim/ug/local-vs-global-optima.html>



개월, 3개월, 6개월 세 기간의 수익률 예측 결과를 도출하였다. 따라서 LSTM과 Transformer 두 모델로 각 다섯 가지 시점에 대해서 세 기간의 예측을 진행하였고 그 결과는 다음과 같다.



Figure 10. 수익률 예측 시점

● LSTM 결과

LSTM 모델의 예측 결과는 장기에서 한계를 보였다. 각 시점 별로 LSTM 모델을 이용하여 예측한 결과는 다음과 같다.

S1시기에서는 모든 기간에서 실제 값보다 높게 예측을 했다. 즉, 상승 모멘텀을 더 강하게 예측했다고 해석할 수 있다. 1개월 예측 결과에서 수익률 변동폭이 실제보다 작지만 급한 상승세가 작은 loss값으로 예측되었다. 3개월과 6개월 예측 결과는 1개월 결과보다 추세를 잘 따라가지만 시간 차가 꽤 크다. Figure 10을 보면 예측 값이 전반적으로 늦게 따라가는 듯한 양상을 보인다.

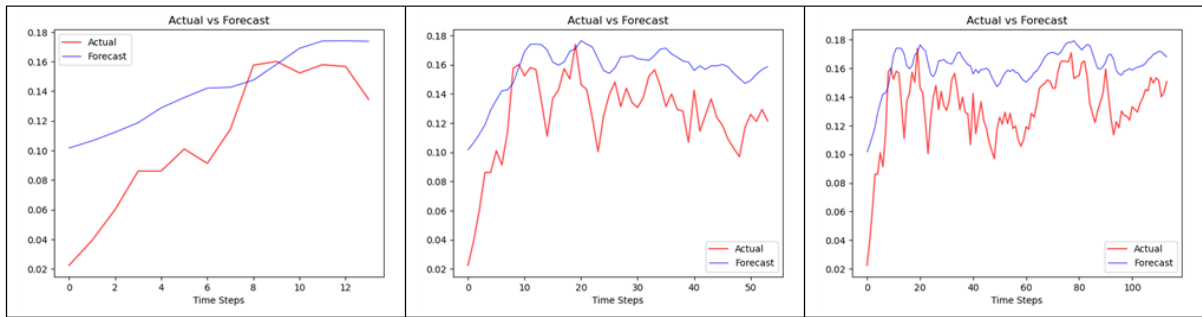


Figure 11. S1 시점의 1개월, 3개월, 6개월의 LSTM 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.0016	0.0013	0.0010

Table 6. S1 시점의 loss

S2시기에서도 S1과 같이 1개월 예측 결과에서 변동폭이 실제보다 작지만 급한 상승세가 작은 loss값으로 예측되었다. 3개월과 6개월 예측 결과도 초반엔 실제 추세와 유사하게 따라갔지만, 시간이 지날수록 실제 값과의 차이가 컸다. 즉, 장기를 예측할수록 예측 값과 실제 값의 큰 간극이 발생했고 그래프 모양도 추세를 따라가지 못했다. Loss도 장기로 갈수록 증가했으며, 1개월 예측보다 6개월 예측 결과의 loss 값은 약 7.7배가 더 높았다.

따라서 S2시기의 장기 예측성은 S1시기에 비해 좋지 않다고 해석할 수 있다. 이는 S1시기가 S2시기와는 달리 예측 시점 전에 이미 상승 추세였기 때문에 보다 급한 상승장을 예측하기 용이했다고 판단된다.

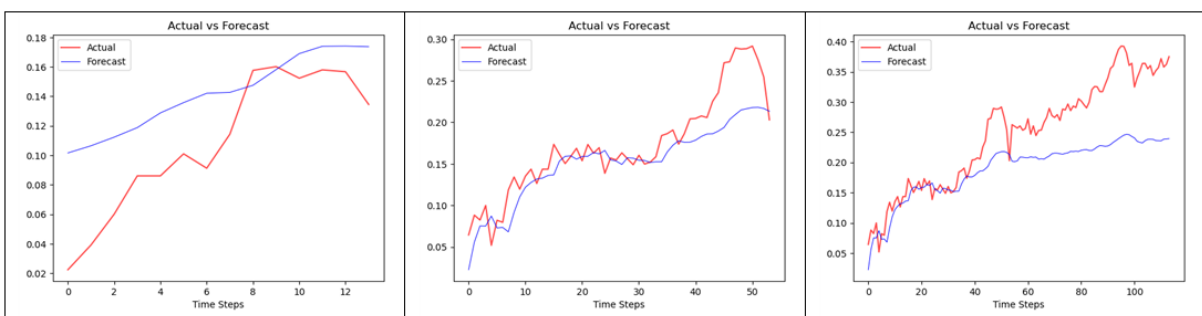


Figure 12. S2 시점의 1개월, 3개월, 6개월의 LSTM 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.0007	0.0011	0.0054

Table 7. S2 시점의 loss

S3시기에서도 1개월 예측 결과의 변동폭은 실제보다 작지만 급한 하락세를 작은 loss값으로 예측했다. 하지만 전체적으로 실제 값보다 예측 값이 높았으며 특히 급락으로 인한 최저점에 대한 예측 값이 실제와 간극이 크게 예측되었다.

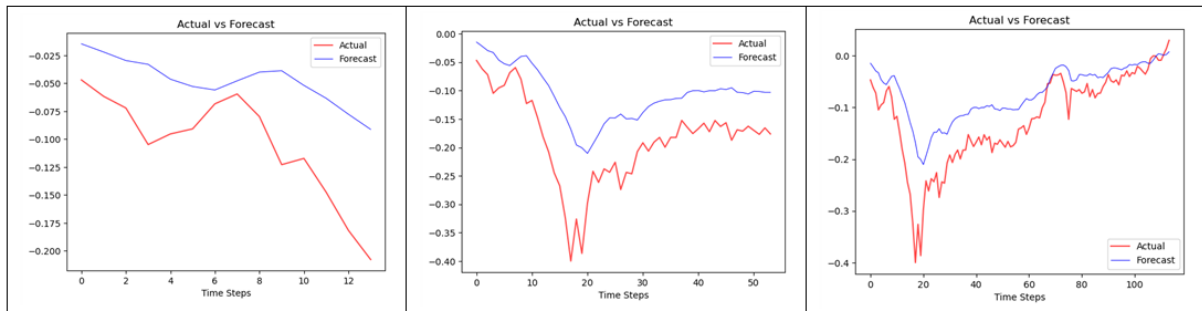


Figure 13. S3 시점의 1개월, 3개월, 6개월의 LSTM 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.0041	0.0077	0.0042

Table 8. S3 시점의 loss

S4시기에서는 loss가 크진 않지만, 다른 시기에 비해 상승하는 추세를 잘 따라가지는 못했다.

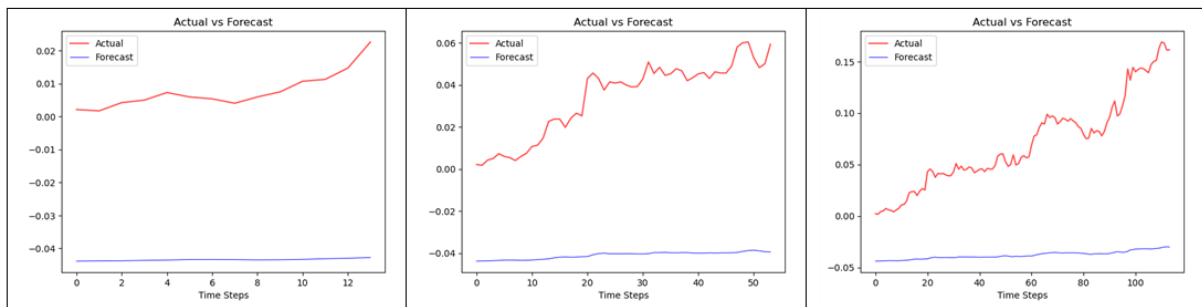


Figure 14. S4 시점의 1개월, 3개월, 6개월의 LSTM 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.0026	0.0058	0.013

Table 9. S4시점의 loss

S5시기의 결과 역시 하락세가 예측됐으나 1개월 예측 결과의 변동폭이 실제보다 작았다. 3개월과 6개월의 예측 결과에서는 시간이 지날수록 실제 값과 예측 값의 간극이 크게 예측되었다.

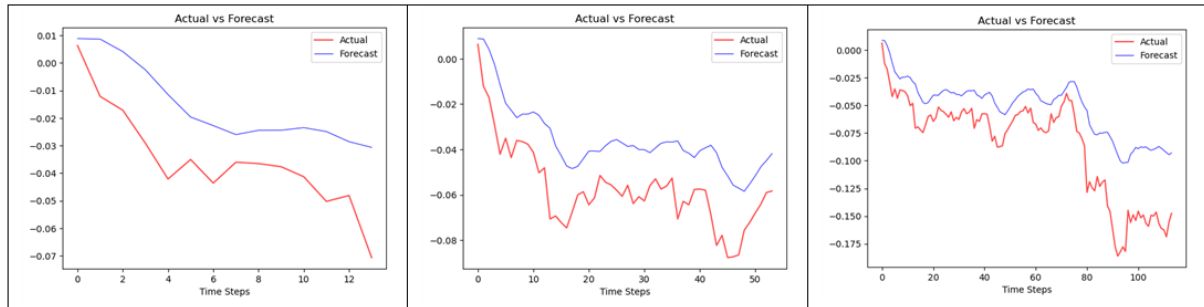


Figure 15. S5 시점의 1개월, 3개월, 6개월의 LSTM 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.0004	0.0005	0.0016

Table 10. S5 시점의 loss

● Transformer 결과

Transformer모델은 LSTM에 비해 단기보다 장기에서 좋은 성능을 보였다. 각 시점 별로 Transformer 모델을 이용하여 예측한 결과는 다음과 같다.

S1시기의 1개월 예측 결과에서 상승세는 예측했지만 실제보다 낮게 예측되었다. 3개월과 6개월 예측 결과에서는 실제보다 변동성이 매우 적은 그래프 모양을 가지고 있다.

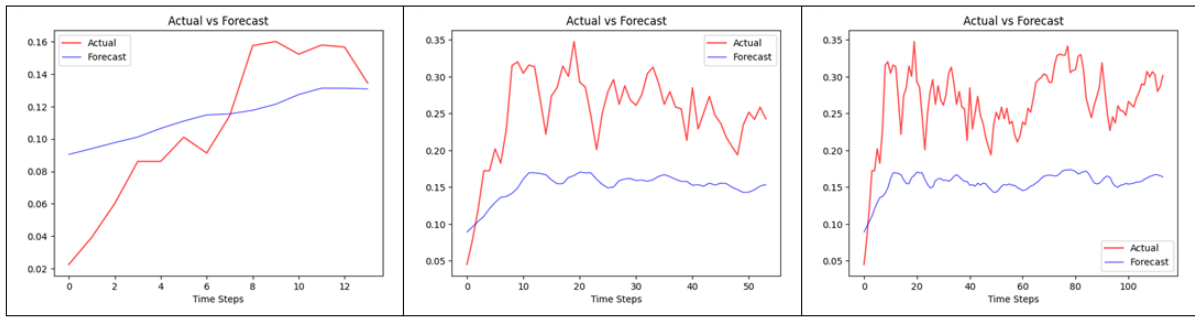


Figure 16. S1 시점의 1개월, 3개월, 6개월의 Transformer 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.00255	0.00914	0.01045

Table 11. S1 시점의 loss

S2시기에서는 S1시기보다 예측 결과가 좋았다. 하락 추세 후의 예측임에도 급한 상승세가 우수하게 예측되었다. 특히 장기로 갈수록 시간 차는 존재하지만, loss가 오히려 감소했으며 실제 값과 가까운 예측 결과를 보여줬다.

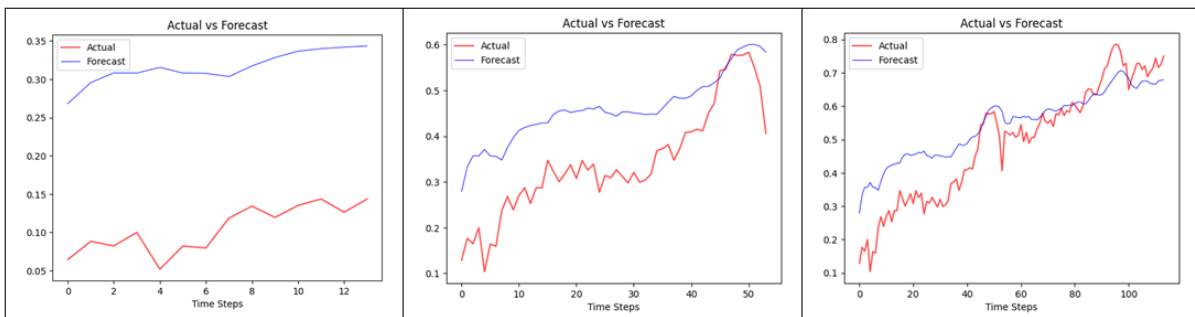


Figure 17. S2 시점의 1개월, 3개월, 6개월의 Transformer 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.03571	0.01238	0.00795

Table 12. S2 시점의 loss

S3시기에서는 1개월 예측 결과의 간극이 매우 컸고 장기로 갈수록 예측 결과가 개선되었다. 시간 차는

있었지만 3개월과 6개월의 예측 결과에서는 급하게 하락 후 다시 상승하는 추세까지 예측되었다. 하지만 급락 지점이 실제보다 높게 예측되었다.

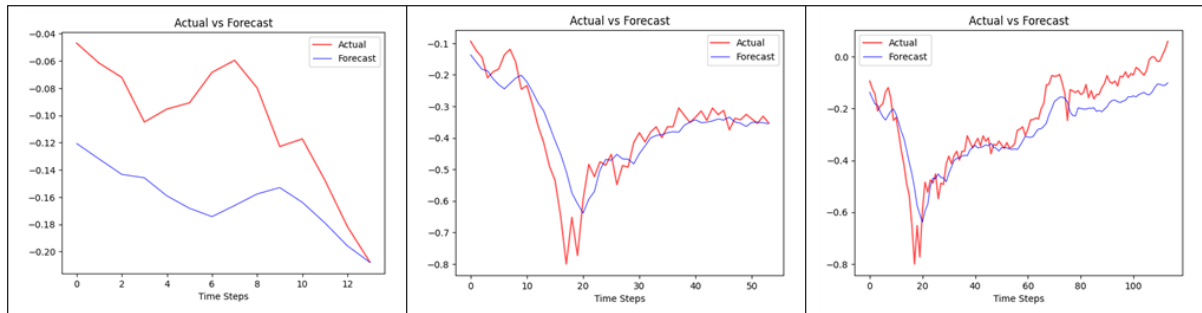


Figure 18. S3 시점의 1개월, 3개월, 6개월의 Transformer 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.00195	0.01761	0.01080

Table 13. S3 시점의 loss

S4시기의 결과에서도 시간차가 있었지만 장기로 갈수록 상승 추세가 우수하게 예측되었다. 그러나 1개월 예측 결과에서는 실제보다 수익률 변동폭이 매우 작았고 실제 값과의 간극도 크게 예측되었다.

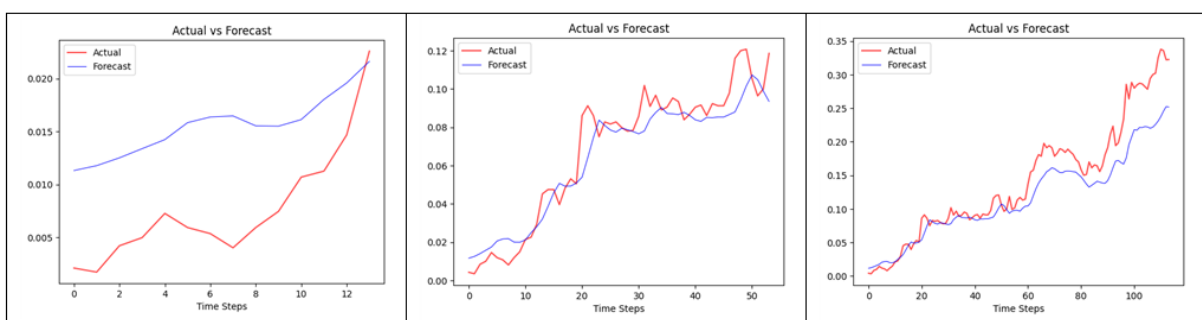


Figure 19. S4 시점의 1개월, 3개월, 6개월의 Transformer 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.00016	0.00047	0.00178

Table 14. S4 시점의 loss

S5시기는 전체적으로 하락 추세가 우수하게 예측되었으나 S4시기에 비해 예측 값과 실제 값의 간극이 크며 실제 값보다 하락 지점이 낮게 예측되었다.

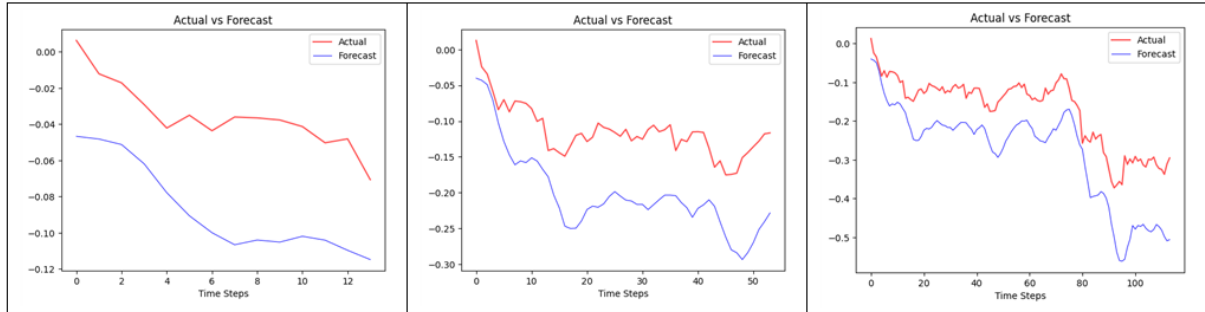


Figure 20. S5 시점의 1개월, 3개월, 6개월의 Transformer 예측 값과 실제 값

	1개월	3개월	6개월
loss	0.00221	0.00538	0.00817

Table 15. S5 시점의 loss

● LSTM, Transformer 비교, 결과 정리

전체 실험 결과에 대한 loss를 정리하면 Table과 같다. Loss는 모든 시기에서 LSTM모델이 우수했다. 이는 window size가 3으로 작기 때문에 실험 환경이 LSTM에 더 유리했기 때문이라고 판단된다.

	LSTM			Transformer		
	1개월	3개월	6개월	1개월	3개월	6개월
S1	0.0016	0.0013	0.0010	0.00255	0.00914	0.01045
S2	0.0007	0.0011	0.0054	0.03571	0.01238	0.00795
S3	0.0041	0.0077	0.0042	0.00195	0.01761	0.01080
S4	0.0026	0.0058	0.013	0.00016	0.00047	0.00178
S5	0.0004	0.0005	0.0016	0.00221	0.00538	0.00817

Table 16. LSTM과 Transformer의 loss

LSTM 모델과 Transformer모델의 실험 결과를 비교했을 때 가장 큰 차이가 났던 시기는 S2시기이다.

LSTM모델은 하락 추세였다가 급격한 상승세를 예측해야 할 때, 즉 급격히 장의 흐름이 바뀔 때 Transformer모델보다 좋지 않은 예측성을 보였다. 특히 장기 예측에 있어서 실제 값과 큰 차이를 보였고 그래프의 추세를 따라가지 못했다. Loss도 LSTM모델은 장기 예측일수록 커진 반면 Transformer는 오히려 줄어들었다.

또한 위의 특수적인 상황을 포함한 전체적인 예측 양상을 고려했을 때 장기 예측은 Transformer모델이 더 우수하다고 판단된다.

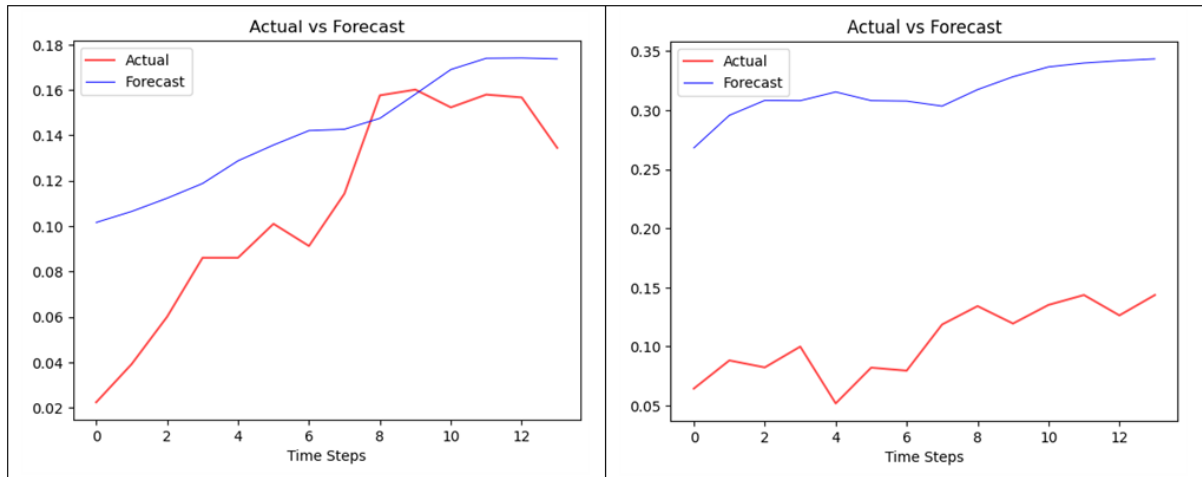


Figure 21. S2시기 1개월 예측 LSTM(좌) Transformer(우)

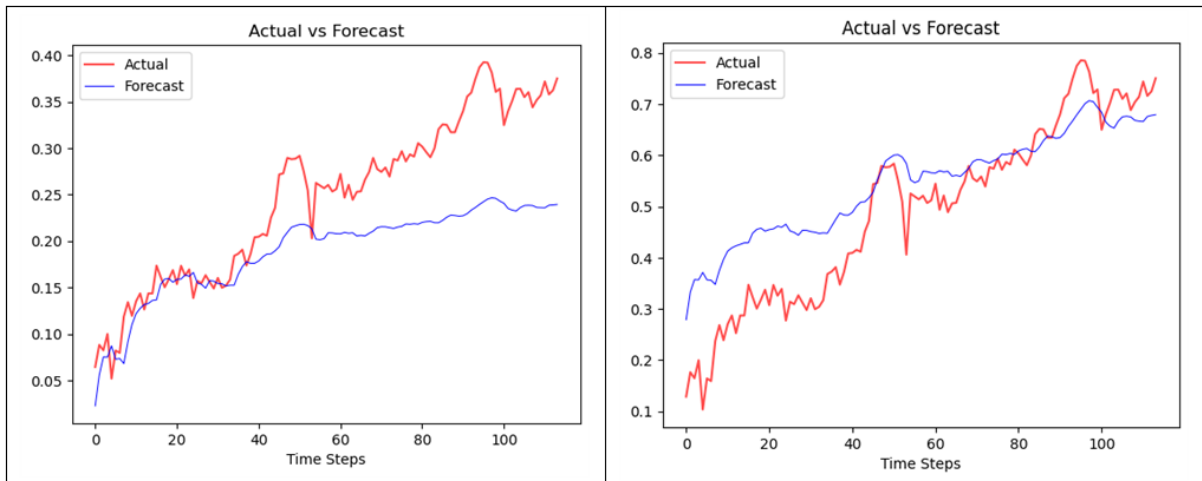


Figure 22. S2시기 6개월 예측 LSTM(좌) Transformer(우)

제 4절 실험의 한계

- Lagging

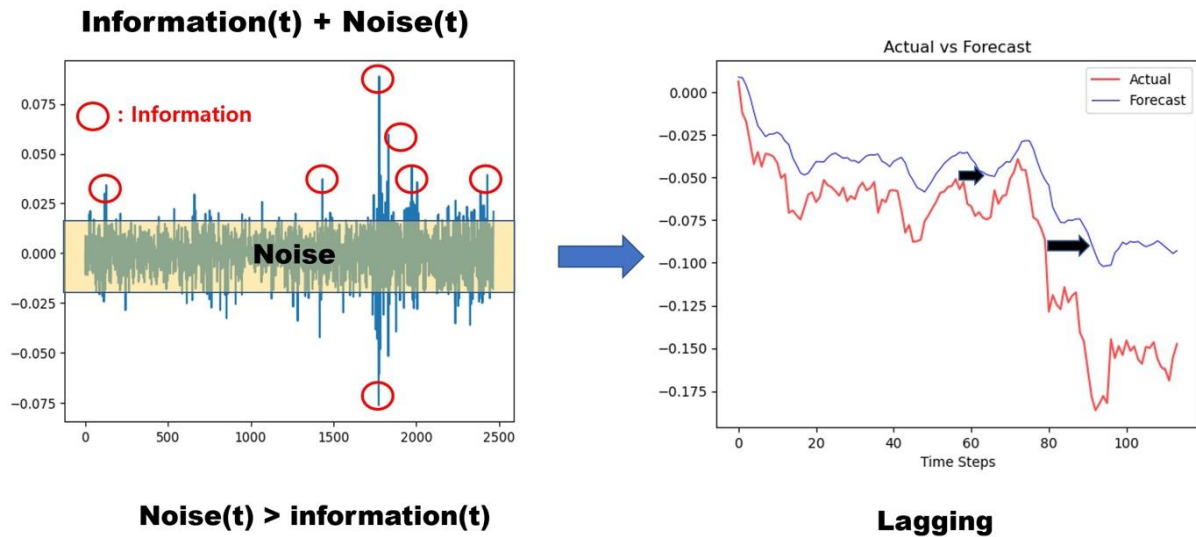


Figure 23. 시계열 데이터 예측의 한계

$$Price_{t+1} = Price_t + Information_t + Noise_t \quad (\text{식 4})$$

시계열 예측에서 자기회귀 모델을 주로 사용하는데, 변수의 과거 값의 선형 조합을 이용하여 관심 있는 변수를 예측한다. 본 연구에서는 자기 회귀 모델 중 AR(1) 모델을 이용하여 위와 같이 모델링을 하였다. 시계열 데이터에는 일반적으로 시간 순차성(Time Step)과 지연 값(Lag)이라는 고유한 2가지 특성이 존재한다. 첫 번째, 시간 순차성(Time Step)은 전구간에서 일정 시간 간격으로 측정된 년, 월, 일, 시간 특성이 다. 두 번째, 지연 값(Lag)은 관측값의 시간 차이로 발생되며 현재 관측값들은 이전 관측값들로 표현된다. $Price(t+1)$ 에 가장 큰 영향을 주는 요소는 $Price(t)$ 이다. 그 다음으로 영향이 큰 요소는 $Noise(t)$ 이고, 간헐적으로 $Information(t)$ 가 나타나서 $Price(t+1)$ 에 영향을 미친다. 좋은 $Information$ 을 $Noise$ 로 인해 잃게 되고, 다음 $Price(t+1)$ 는 현재 $Price(t)$ 와 $Noise(t)$ 로 구성된 것처럼 설명되기에 예측 모델이 잔차를 최소화 하기 위해 후행 예측(Lagging)을 야기한다. 이런 이유로 주가 예측을 시도할 때 멀리서 보면 잘 맞추는 것 같아 보이지만, 가까이서 보게 되면 Lagging되는 형태로 예측값이 나온다.

- 학습

본 실험에서는 조기종료(Early Stopping), 모델 저장(Model Saving), Epochs 및 Iteration 등에 대한 다양한 학습을 진행해보지 못했다는 한계가 있다. 우선, 조기 종료는 과적합이 일어나기 전 훈련을 멈춤으로써 정규화 효과를 주어 과적합을 방지하기 위한 기법이다. 과적합이 발생하기 전까지는 학습에 대한 오차와 검증에 대한 오차가 모두 감소하지만, 과적합이 발생하면 학습 데이터셋에 대한 오차만 감소하고 검증 데이터셋에 대한 오차가 증가한다. 따라서 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정하는 것이다.

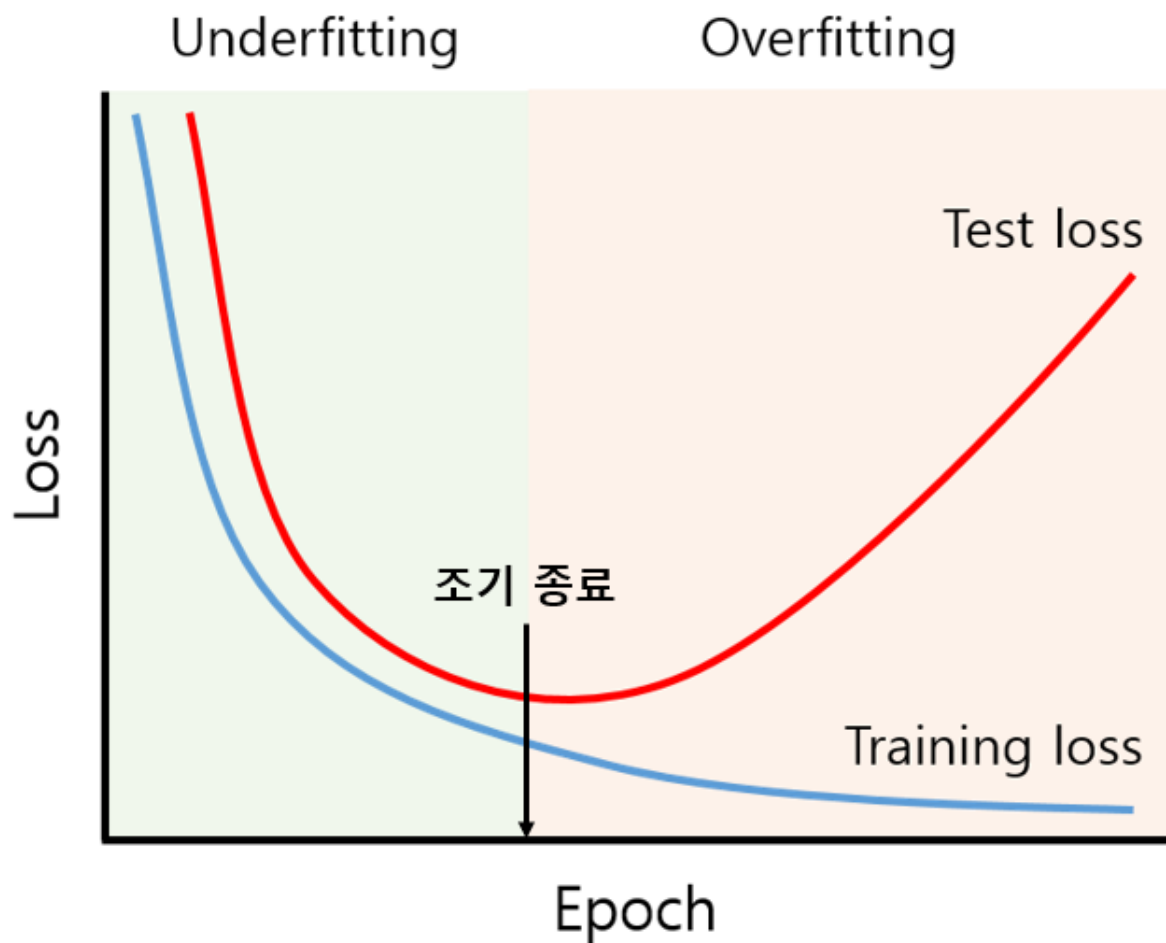


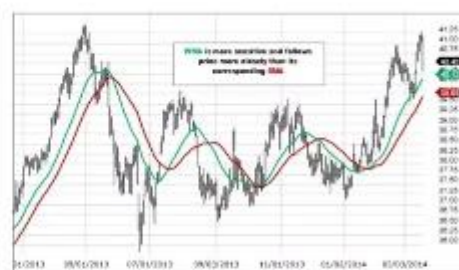
Figure 24. 조기종료

한편, 모델을 저장하고 복구하는 데에는 상당한 노력과 시간이 필요하다. 그러나 모델 저장은 모델을 학습시키는 도중에 어떤 문제가 발생하여 작업이 중단돼도 저장된 모델을 불러와 해당 부분부터 시작할 수

있고, 코드를 공유할 수 있게 함으로써 모델의 정확도와 효율성을 향상시키며 더 나은 모델을 만들 수 있다. 마지막으로 Epoch란 전체 데이터셋을 학습한 횟수를 의미하며, Iteration은 1 Epoch를 마치는 데 필요한 파라미터 업데이트 횟수를 의미한다. Epoch가 높을수록 적합한 파라미터를 찾을(loss 감소) 확률이 올라가지만, 지나치게 높게 설정한다면 과적합이 발생한다. 적절한 Epoch와 Iteration을 설정함으로써 과소적합(Underfitting)과 과적합을 방지할 수 있다.

제 4장 결론 및 한계점

1. Moving Average (MA, EMA, ...)



2. Bilateral Filter

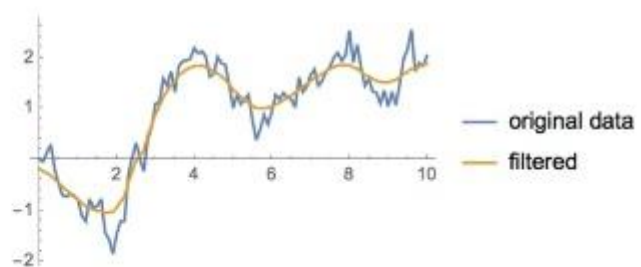


Figure 25. Time-series denoising

딥러닝은 다층 신경망으로 구성되어 있으며, 많은 수의 뉴런과 layer로 이루어져 있다. 이런 구조는 모델의 복잡성을 증가시키고 설명력을 저하시킨다. 딥러닝 모델은 블랙 박스로 간주되기도 하는데, 입력과 출력 간의 관계를 해석하기 어려워 이로 인하여 예측에 영향을 미치는 요인이 불분명하고, 내부 동작을 해석할 수 있는 가능성이 떨어진다. 하지만 위 실험을 통하여 볼 수 있듯이, 단순한 모델을 사용함에도 좋은 예측 성능을 발휘한다. 이는 딥러닝 모델이 합리적인 투자 결정을 내리는 데 기여할 수 있음을 보여주고, 딥러닝 모델의 유의성을 보여준다.

본 실험에서는 *SONG, Xuanyi, et al.(2020)*에서 진행된 딥러닝 모델을 활용하여 시계열 데이터를 예측한 연구를 기반으로 삼아 LSTM모델과 Transformer모델을 사용하여 금융 시계열 데이터로 실험을 하였다. LSTM모델은 긴 기간보다 짧은 기간에서의 예측 성능이 좋았고, 미래의 주가 변동이 과거 추세와 유사한 동향을 보일 때 예측력이 우수했다. Transformer모델은 긴 기간의 데이터를 학습하여 보다 장기적인 미래를 예측할 때에 유리하다. 특히 Transformer모델은 학습 기간과 예측 기간의 추세의 흐름이 바뀌는 상황에 대한 예측 성능이 더 우수했다. 이는 Transformer모델이 자연어처리 분야에서만 유용한 모델이 아니라는 점을 보여준다. 다만, 본 연구에서는 Feature를 하나밖에 고려하지 못했다. 추후에 진행될 연구에서는 *QIN, Yao, et al.(2017)*에서 진행된 단일 Transformer 모델의 1차원 적용 문제를 Dual-Stage Attention 같은 아이디어 연구를 차용하여 다양한 Feature를 적용해 비교 분석한다. Dual-Stage-attention의 전체 구조는 Input Attention + 인코더 + 디코더로 이루어져 있다. Input Attention에서 여러 외생 변수들 사이에서 인코더 hidden state를 이용하여 중요한 feature를 뽑아낸다. 그렇게 뽑은 데이터를 인코더의 input으로 사용한다. 인코더의 hidden state를 업데이트할 때도 디코더의 hidden state를 사용한다. 그리고 시계열 데이터의 노이즈를 이동평균선 또는 Bilateral Filter(양방향 필터)를 통해 줄인다(*KÖHLER et al. 2005*). 모델의 기술적인 문제를 개선한 뒤 다음과 같이 모델을 확장 연구한다.

제 1절 후속 연구

미국 시장 데이터를 이용한 모델

미국 시장의 경우 국내 시장에 비해 다양한 장점이 있는데, 먼저 시장의 크기와 유동성이다. 미국 주식 시장은 세계에서 가장 크고 유동성이 높은 주식 시장 중 하나로 다양한 종목을 보유하고 있으며 거래량 역시 많다. 또한 다양한 트렌드와 패턴을 포함하고 있어 딥러닝 모델의 학습과 예측에 유리하다. 다음으로 글로벌 시장에 큰 영향을 미치는 주요 시장 중 하나라는 점이다. 미국 주식 시장의 흐름 및 동향은 세계적으로 많은 투자자와 시장 참여자들에게 영향을 미치고 있다. 이는 곧, 미국 주식 시장에서 딥러닝 모델을 개발하고 검증할 경우에 국제적인 관점과 영향을 고려할 수 있다는 장점으로 이어진다.

주가 예측 + 뉴스 감성 분석 모델

주식의 가격을 이해하고 예측하기 위해서 활용되는 데이터의 범위는 기존의 정형화된 데이터에서 비정형화 된 다양한 종류의 데이터까지 확대되고 있다. 비정형화 데이터인 텍스트 데이터를 정형 데이터인 가격 데이터와 결합해 모델을 구축하고 텍스트 데이터를 감성 분석에 이용한다. ‘감성 분석(Sentiment Analysis)’이란 자연어처리 기술을 사용하여 문장 혹은 문서에서의 표현이 긍정, 부정 혹은 중립인지를 판별하는 기술이다. 이는 주로 BERT와 같은 Transformer 기반 언어 모델을 통하여 이루어진다. *DAY et*



al.(2016)에 의하면 일정 기간의 뉴스 데이터를 수집하여, 뉴스 데이터에 대한 긍정, 부정, 중립 감성 분석을 진행한다.³ 또한, 분석 데이터가 일정 시간 후의 가격 변동의 방향과 가격 변동의 폭에 대한 예측력을 가지는지 조사한다.

³ DAY, Min-Yuh; LEE, Chia-Chou. Deep learning for financial sentiment analysis on finance news providers. In: *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2016. p. 1127-1134.



참고문헌

- [1] 김동건; 김광수. 시계열 예측을 위한 스타일 기반 Transformer. *정보처리학회논문지/소프트웨어 및 데이터 공학 제*, 2021, 10.12: 12.
- [2] 김수현. LSTM 기반 모형의 주식시장 예측성 분석. *Journal of The Korean Data Analysis Society*, 2020, 22.5: 1989-2000.
- [3] DAY, Min-Yuh; LEE, Chia-Chou. Deep learning for financial sentiment analysis on finance news providers. In: *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2016. p. 1127-1134.
- [4] HOCHREITER, Sepp; SCHMIDHUBER, Jürgen. Long short-term memory. *Neural computation*, 1997, 9.8: 1735-1780.
- [5] KÖHLER, Torsten; LORENZ, Dirk. A comparison of denoising methods for one dimensional time series. *Bremen, Germany, University of Bremen*, 2005, 131: 950.
- [6] QIN, Yao, et al. A dual-stage attention-based recurrent neural network for time series prediction. *arXiv preprint arXiv:1704.02971*, 2017.
- [7] SHI, Jimeng; JAIN, Mahek; NARASIMHAN, Giri. Time series forecasting (tsf) using various deep learning models. *arXiv preprint arXiv:2204.11115*, 2022.
- [8] SONG, Xuanyi, et al. Time-series well performance prediction based on Long Short-Term Memory (LSTM) neural network model. *Journal of Petroleum Science and Engineering*, 2020, 186: 106682.
- [9] VASWANI, Ashish, et al. Attention is all you need. *Advances in neural information processing systems*, 2017, 30.

